

12. Computer Arithmetic

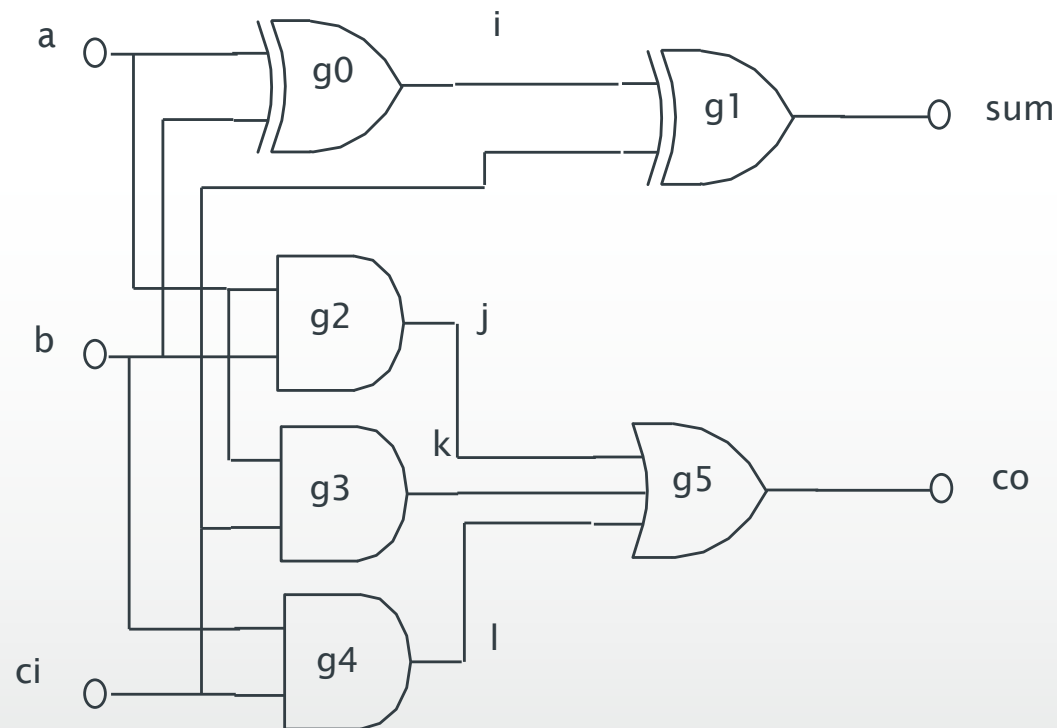
– Adders

Integers

- Unsigned integers are represented as bits, with the most significant bit at the left
- `logic [7:0] a, b;`
- Adding 2 integers means adding `a[0]` to `b[0]`, generating a sum bit and a carry bit
 - Repeat for bits 1, 2, etc.
 - May generate a carry bit

Full Adder

- Basic Building Block
 - 1-bit Adder with Carry-in and Carry-Out



A	B	Ci	S	Co
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Half Adder

- No Carry-in
Sum = $a \oplus b$
Co = $a.b$
- 2 gates
- Used in, e.g. multipliers. See later.

Structural SystemVerilog

```
module full_adder (output logic sum, co,  
                  input logic a, b, ci);  
  
    logic i, j, k, l;  
  
    xor g0 (i, a, b);  
    xor g1 (sum, i, ci);  
    and g2 (j, a, b);  
    and g3 (k, a, ci);  
    and g4 (l, b, ci);  
    or g5 (co, j, k, l);  
  
endmodule
```

Dataflow SystemVerilog

```
module full_adder (output logic sum, co,  
                  input logic a, b, ci);
```

```
always_comb
```

```
begin
```

```
sum = a ^ b ^ ci;
```

```
co = (a & b) |
```

```
      (a & ci) |
```

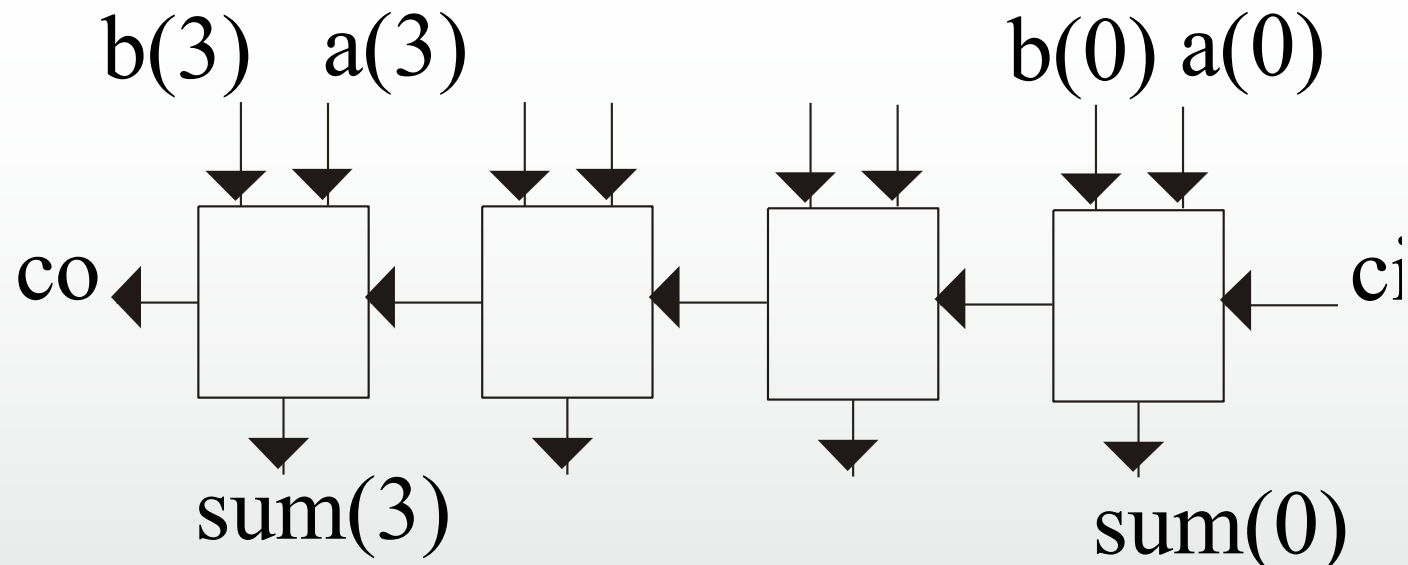
```
      (b & ci);
```

```
end
```

```
endmodule
```

Hierarchical Designs

- Four bit adder built from full adder
- c_i is 0



Hierarchical Designs

- Complete 4 bit adder (SystemVerilog):

```
module four_bit_adder
    (output logic [3:0] sum, output logic co,
     input logic [3:0] a, b, input logic ci);

    logic [3:1] carry;

    full_adder fa0 (sum[0], carry[1], a[0], b[0], ci);
    full_adder fa1 (sum[1], carry[2], a[1], b[1], carry[1]);
    full_adder fa2 (sum[2], carry[3], a[2], b[2], carry[2]);
    full_adder fa3 (sum[3], co, a[3], b[3], carry[3]);

endmodule
```


Parameterised Ripple Adder

```
module ripple_adder #(parameter N = 4)
  (output logic [N-1:0] Sum, output logic Cout,
   input logic [N-1:0] A, B, input logic Cin);

  logic [N-1:1] Ca;
  genvar i;

  fulladder f0 (Sum[0], Ca[1], A[0], B[0], Cin);

  generate for (i = 1; i < N-1; i++)
    begin : f_loop
      fulladder fi (Sum[i], Ca[i+1], A[i], B[i], Ca[i]);
    end
  endgenerate

  fulladder fN (Sum[N-1], Cout, A[N-1], B[N-1], Ca[N-1]);

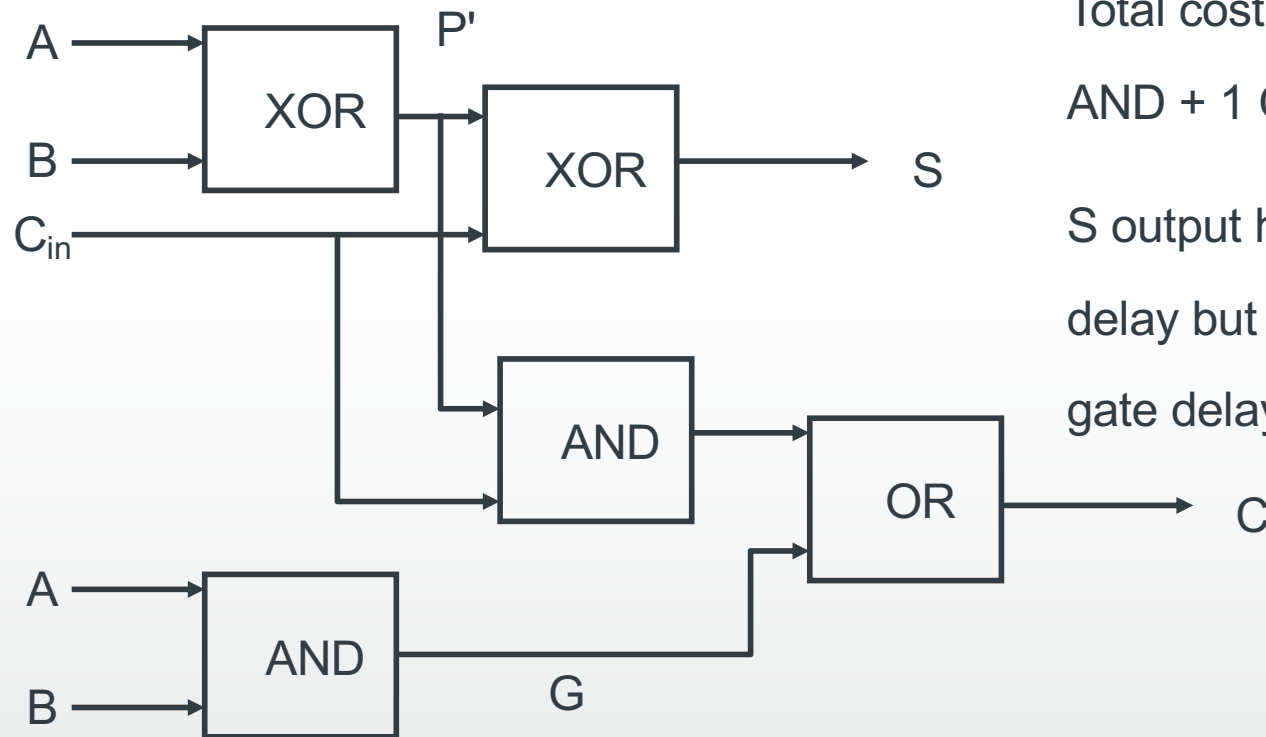
endmodule
```

0 and N are special cases

Delay through ripple adder

- $T_d \text{ (worst)} = (N-1)T_{\text{carry}} + T_{\text{sum}}$
- T_{carry} is delay through AND/OR or NAND/NAND
- T_{sum} is delay through XOR
- Ignore delay through wires
- Linear increase in delay with number of bits

Redraw full adder



Save a gate

Total cost: 2 XOR + 2

AND + 1 OR

S output has two gate
delay but C has three
gate delay

Expanded Truth Table

C_{in}	A	B	$A.B$ (G)	$A+B$ (P)	$A \oplus B$ (P')	S	C
0	0	0	0	0	0	0	0
0	0	1	0	1	1	1	0
0	1	0	0	1	1	1	0
0	1	1	1	1	0	0	1
1	0	0	0	0	0	1	0
1	0	1	0	1	1	0	1
1	1	0	0	1	1	0	1
1	1	1	1	1	0	1	1

Generate/Propagate Carry

- Generate function: The generate signal G is true when C_O is generated internally within the adder.

$$G = A \cdot B$$

- Propagate function: If the propagate signal P is true then C_{in} is passed to C_O when C_{in} is true
- $P = A + B$; but we can also use $P' = A \oplus B$ since it shows the same behaviour and can be used to generate the sum term

$$S = P \oplus C_{in}$$

$$C = G + P \cdot C_{in}$$

Carry-Lookahead Adder (CLA)

Idea

- Try to “predict” C_k earlier than $T_{carry} * k$
- Instead of passing through k stages, compute C_k separately using 1-stage CMOS logic
- Look at the 0th stage carry and predict all the carry corresponding to the different stages
- Use Propagate-Generate (PG) formula

CLA: formulation of carry propagation equation

If $C_4=1$, then either:

g_3

generated at bit pos 3

$g_2 \cdot p_3$

generated at bit pos 2, propagated 3

$g_1 \cdot p_2 \cdot p_3$

generated at bit pos 1, propagated 2,3

$g_0 \cdot p_1 \cdot p_2 \cdot p_3$

generated at bit pos 0, propagated 1,2,3

$C_{in} \cdot p_0 \cdot p_1 \cdot p_2 \cdot p_3$

input carry, propagated 0,1,2,3

$$C_4 = g_3 + g_2 \cdot p_3 + g_1 \cdot p_2 \cdot p_3 + g_0 \cdot p_1 \cdot p_2 \cdot p_3 + C_{in} \cdot p_0 \cdot p_1 \cdot p_2 \cdot p_3$$

Similarly:

$$C_3 = g_2 + g_1 \cdot p_2 + g_0 \cdot p_1 \cdot p_2 + C_{in} \cdot p_0 \cdot p_1 \cdot p_2$$

$$C_2 = g_1 + g_0 \cdot p_1 + C_{in} \cdot p_0 \cdot p_1$$

$$C_1 = g_0 + C_{in} \cdot p_0$$

CLA: Summary

CLA compared to ripple-carry adder:

- 2-level logic to generate C_4
- Delay is approx T_{carry} but increases with bigger gates
 - Faster (“4 times”?), but delay still linear (w.r.t. # of bits, see next slide)
- Larger area
 - P, G signal generation
 - Carry generation circuits
 - Carry generation circuit for each bit position (no re-use)

Limitation: cannot go beyond 4 bits of look-ahead

- Large p,g fan-out slows down carry generation

N-bit CLA architecture

